

MediSync

Django migration: stage-by-stage guide

What each stage builds, how to tell it is finished, and what to test.

Written for the team. Status as of October 3, 2026.

The project in one paragraph

MediSync is a student clinic app. Students book a doctor or dentist, answer a short triage questionnaire, get a queue number, and follow their place in line. We are converting a working single-file HTML prototype (**medisync-prototype.html**) into a normal Django application. The prototype is the source of truth for how every student screen looks and reads. We keep its content, wording, layout, icons and buttons, and replace its fake in-memory data with real models, views, forms and database queries.

Status at a glance

Stage	Name	Status
0	Project skeleton	Done
1	Layout shell and dashboard	Done (dashboard still shows sample data)
2	Accounts: login, profile, settings	Done (21 tests passing)
	Placeholder pages so every tab opens	Done (temporary, uses sample data)
3	Clinic models, admin, seed data, slot generator	Next
4	Booking steps 1 to 3	Not started
5	Questionnaire and triage	Not started
6	Confirmation, appointments, notifications	Not started
7	Live queue, polling, cancel, staff call-next	Not started
8	History, dashboard, notifications, help	Not started
9	Error pages, security, tests, production	Not started

1. Ground rules

- **Do not restart the architecture.** The structure below is decided. Change it only after the team agrees.
- **Keep the prototype's UI intact.** Same text, cards, buttons, modals, icons and styling. A button that only showed a toast in the prototype becomes a real feature, it does not get deleted.
- **Stack:** Django 5.2 ($\geq 5.2, < 5.3$), django-environ, Django templates and vanilla JavaScript. No DRF, Celery, Channels, Tailwind, HTMX or JS framework.
- **clinic = what exists. appointments = what the student does with it.** Do not add extra apps for queue or triage.
- **Validate on the server** for anything that matters, and back it with database constraints.
- **Every student view requires login** and every appointment lookup is filtered by `request.user`.
- **JavaScript only for UI behaviour:** sidebar, dropdowns, modals, toast dismissal, live questionnaire feedback, queue polling.
- **Timezone is Asia/Manila.** Use Django's timezone-aware local date, never a hardcoded date.
- **Staff use Django admin.** There is no custom staff dashboard unless the team asks for one later.
- **No public signup.** Accounts come from admin or the seed command.

Every stage must pass this checklist before it counts as done

- `python manage.py check` reports no issues.
- `python manage.py makemigrations --check` reports no missing migrations, and the migration files are committed.
- `python manage.py test apps` passes, including new tests for the stage.
- The new pages work in a browser at desktop and phone width, and look like the prototype.
- Nothing for this stage still reads from `apps/core/mock.py`.

2. Apps and what each one owns

App	Owns	Does not own
core	Layout, dashboard, help and FAQ, error pages, template tags, icons, toast helper	Any clinic or appointment data
accounts	Custom User, StudentProfile, UserSettings, login, logout, password change, data export	Appointments
clinic	Practitioner, PractitionerAvailability, VisitReason, Symptom, ClosedDate, slot generation	What a student booked
appointments	Booking flow, Appointment, TriageAssessment, triage scoring, queue state, history, cancellation	The list of practitioners and reasons
notifications	Notification model, dropdown, SMS and email service abstraction (console only in development)	Deciding when to notify (appointments calls it)

3. Data model

Appointment is the central model. Visit history is a query on Appointment, it is never stored separately.

Model	App	Purpose and key fields
User	accounts	Custom AbstractUser, created before the first migration. Login identity, name, email.
StudentProfile	accounts	One per user. Student ID (unique, may be blank), course, year level, phone, blood type, allergies, notes, emergency contact.

Model	App	Purpose and key fields
UserSettings	accounts	One per user. SMS alerts, email reminders, queue sound, auto-start triage, share anonymous stats.
Practitioner	clinic	Doctor or dentist: name, role, type, room, active flag.
PractitionerAvailability	clinic	Working windows per practitioner, used to generate slots.
VisitReason	clinic	Reason for visit, visit type, triage weight .
Symptom	clinic	Questionnaire item, weight, red-flag field.
ClosedDate	clinic	Dates the clinic is closed. No slots are generated.
Appointment	appointments	patient, practitioner, reason, date, start_time, status, priority, queue_number, reference, room.
TriageAssessment	appointments	One per appointment. Severity, description, selected symptoms, consent, score, resulting priority.
Notification	notifications	user, title, body, created time, read flag, optional link.
FAQ	core	Question, answer, display order, active flag.

Appointment statuses: confirmed, called, checked_in, completed, cancelled, no_show. **Priority:** low, medium, high.

Double booking is blocked by the database: a conditional UniqueConstraint on practitioner, date and start_time, applied only where the appointment is not cancelled. A cancelled slot can be booked again.

4. Pages and URLs

URL	Page	Stage
/	Dashboard	1, 8
/accounts/login/ and /accounts/logout/	Sign in, sign out (sign out is a POST)	2
/accounts/password/	Change password	2
/profile/ and /profile/emergency/	Medical profile, emergency contact	2
/settings/ and /settings/toggle/	Settings and switches	2
/export/	Download my data (JSON)	2, 8
/book/	Booking start: visit type and reason	4
/book/practitioner/	Choose practitioner	4
/book/schedule/	Choose date and slot	4
/book/symptoms/	Triage questionnaire	5
/book/result/	Triage result and estimated wait	5
/book/confirm/	Confirm booking (POST creates the appointment)	6
/appointments/<ref>/	Appointment confirmation	6
/appointments/<ref>/cancel/	Cancel (POST)	7
/queue/ and /queue/status/	Queue page and JSON polling endpoint	7
/history/	Visit history	8
/help/	Help and FAQ	8
/admin/	Staff interface	0, then every stage

The placeholder pages added after Stage 2 use slightly different URLs for confirmation (/appointments/confirmation/). Stage 6 replaces that with the reference-based URL above.

5. Stage details

Stage 0: Project skeleton

DONE

GOAL

A Django project that runs, with the folder layout, settings split and custom User in place before any real feature exists.

WHAT TO BUILD

- Project package config/ with settings/base.py, dev.py, prod.py, urls, wsgi, asgi.
- Five apps under apps/: core, accounts, clinic, appointments, notifications.
- Custom accounts.User (AbstractUser) set as AUTH_USER_MODEL before the first migration. Swapping it later is painful.
- django-environ reads .env. example.env shows the variables with placeholder values.
- Requirements split into base.txt, dev.txt, prod.txt. .gitignore excludes .env, the database and the venv.

DONE WHEN

- The server starts and /admin/ loads.
- Migrations apply on a fresh database.

TESTS

- No feature tests yet. manage.py check is the test for this stage.

NOTES FOR THE TEAM

- Never commit .env, db.sqlite3 or any file with credentials.

Stage 1: Layout shell and dashboard

DONE

GOAL

Move the prototype's frame (sidebar, top bar, toasts, base styles) into Django templates so every later page just fills a content block.

WHAT TO BUILD

- templates/base.html with a replaceable shell block (login page replaces it).
- Partials: sidebar, topbar (notifications dropdown, user menu), toasts.
- Components: button, badge, empty_state. Later stages add card, stepper, choice_card, modal, form_field.
- Icon tag {% icon "bell" 18 %} backed by the prototype's SVG data in apps/core/icons.py. Unknown names fall back to the info icon.
- CSS split into tokens, base, layout, components, forms, tables, feedback, responsive, plus one file per page under static/css/pages/.
- JS modules: sidebar toggle, dropdowns, toast auto-dismiss, loaded as ES modules from main.js.
- Dashboard rendered by Django from sample data in apps/core/mock.py.

DONE WHEN

- The dashboard looks like the prototype at desktop and phone width.
- Sidebar highlights the current page and works as a drawer on mobile.

TESTS

- Shell renders for a logged-in user (covered in the accounts test suite).

NOTES FOR THE TEAM

- The prototype's class name bugs are fixed here: btn-bp became btn--primary style modifiers, and the two meanings of .cnt were split.

GOAL

Replace the hardcoded student with real users. Students sign in, edit their medical profile, and control their settings.

WHAT TO BUILD

- **Models:** StudentProfile and UserSettings, created automatically for every user by a signal. Defaults match the prototype (SMS on, email on, sound off, auto-triage on, anonymous stats off).
- **Login and logout:** login page in the prototype's card style. Sign out is a POST from a confirmation modal.
- **Medical profile:** edits name and email on User and everything else on StudentProfile. Validates blood type (A+, A-, B+, B-, AB+, AB-, O+, O-), phone format and unique student ID. Emergency contact has its own modal and form.
- **Settings:** each switch is a tiny POST form, so there is no client-side state. The server flips the value.
- **Account actions:** change password (Django's form), download my data (JSON of the student's own profile and settings).
- **Admin:** profile and settings editable inline on the user page.
- **Components added:** {% modal %} tag, form_field, toggle, toast helper. Toasts keep the prototype's title plus body format.
- **Seed command:** `python manage.py seed_demo` recreates the prototype's demo student (Alex Mendoza).

DONE WHEN

- Every student page redirects to login when signed out.
- A student can only read and change their own profile and settings.
- Migration 0003 backfills profiles for users created before this stage.

TESTS

- 21 tests: signals and defaults, login and logout, logout needs POST, profile validation, unique student ID, blank IDs do not collide, cannot edit another student's data, toggles only affect the owner, export contains own data only.

NOTES FOR THE TEAM

- Login uses a username for now. Accepting the student ID as well is a small change if the team wants it.
- The prototype's fake 'reset link sent' toast became a real change-password form.

Placeholder pages (added after Stage 2)

So every sidebar tab opens, Queue Status, Visit History, Help and the booking steps have temporary pages that show the prototype's sample data or a 'coming in Stage N' card. They exist only to make navigation work. Each one is deleted or rewritten by the stage that owns it. **Do not build on the placeholder views.**

GOAL

Move everything the clinic offers (practitioners, reasons, symptoms, hours, closures) out of hardcoded lists and into the database, and generate bookable slots from it.

WHAT TO BUILD

- **Models in clinic:** Practitioner, PractitionerAvailability, VisitReason (with triage weight), Symptom (with red-flag field and weight), ClosedDate.
- **Admin:** register every model with sensible list columns, filters and search. Staff manage the clinic here.
- **Slot generator** as a pure function in `apps/clinic/services/slots.py`. Default windows from the prototype: 9:00 to 11:30 and 13:00 to 16:00, every 30 minutes. It takes a practitioner, a date and a set of already-taken times, and returns the open slots.
- The generator skips closed dates and times already in the past (compared with Manila local time).
- **seed_demo extended:** create the practitioners, visit reasons and symptoms from the prototype's PRAC, REASONS and FLAGS lists, plus default availability.
- Migrations committed.

DONE WHEN

- A staff user can add and edit all clinic data in admin.
- `seed_demo` gives a database that matches the prototype's options.
- Slots for a normal weekday match the prototype's grid.

TESTS

- Slot generation: default grid, closed date returns nothing, taken slots removed, past slots removed, a practitioner with no availability returns nothing.
- Model constraints and string representations.

NOTES FOR THE TEAM

- The slot generator takes the set of booked times as an argument so it stays pure. Stage 4 supplies that set from Appointment.
- Keep the generator free of Django imports where practical so tests run fast.

Stage 4: Booking steps 1 to 3

NOT STARTED

GOAL

Let a student choose a visit type and reason, a practitioner, and a date and time. Nothing is saved as an appointment yet.

WHAT TO BUILD

- Booking state lives in the **session**: type, then reason, then practitioner, then date and slot, then answers.
- Views for `/book/`, `/book/practitioner/`, `/book/schedule/`, replacing the placeholders.
- Step guard: if an earlier step is missing, redirect to the first incomplete step.
- Server-side validation: the practitioner must offer the chosen type, the date must be open and not in the past, and the slot must be in the generated list.
- Components: stepper, choice_card, card. The calendar and slot grid follow the prototype's look.
- Discard-booking modal: clears the session booking only. It never touches saved appointments.
- The slot hold is display-only. There is no real reservation lock yet.

DONE WHEN

- A student can walk through all three steps and reach the questionnaire page.
- Tampering with form values or URLs cannot select an invalid practitioner, date or slot.

TESTS

- Step guard redirects, invalid slot rejected, closed date rejected, past date rejected, practitioner of the wrong type rejected, discard clears the session.

NOTES FOR THE TEAM

- The prototype bug where discarding could cancel the previous confirmed appointment (`cancelprio`) must not exist here. Discard only clears session data.

Stage 5: Questionnaire and triage

NOT STARTED

GOAL

Collect symptoms and severity, score them, and show the student their priority and estimated wait.

WHAT TO BUILD

- `/book/symptoms/`: severity slider, symptom checkboxes (including red flags), description with character count, consent checkbox.
- **Scoring service** in `apps/appointments/services/triage.py`: a pure function. Inputs are the reason's weight, severity and selected symptom weights. Output is a score and a priority (low, medium, high).
- Any red-flag symptom forces high priority.
- `TriageAssessment` model, linked to the appointment when it is confirmed.
- `/book/result/`: shows the priority, estimated wait and what happens next. The line 'Triage is queue-priority support only, not a diagnosis' stays visible.
- Small vanilla JS for live feedback: severity label and colour, character counter.

DONE WHEN

- Priority is computed on the server. The browser never decides it.
- The result page matches the prototype's wording.

TESTS

- Table-driven scoring tests, red-flag override, threshold boundaries, empty symptom list, maximum severity, missing consent rejected.

NOTES FOR THE TEAM

- Keep the thresholds as named constants in one place so clinic staff can review them.

Stage 6: Confirmation, appointments, notifications

NOT STARTED

GOAL

Turn the session booking into a real Appointment, safely, and tell the student about it.

WHAT TO BUILD

- Confirm view: a POST that creates the Appointment and TriageAssessment inside `transaction.atomic()`.
- Queue number and reference are assigned inside the transaction. The reference is a short unique string used in URLs.
- If two students take the same slot, the database constraint raises an error. The view catches it and shows 'that slot was just taken' with a way back to the schedule.
- `/appointments/<ref>/`: confirmation page, filtered by `request.user`. Another student's reference returns 404.
- Notification row created on booking. SMS and email go through a service abstraction. In development they print to the console. A real provider can plug in later.
- Add to calendar: downloads an .ics file. Resend SMS: sends again through the same service.

DONE WHEN

- Confirming creates exactly one appointment, or nothing if anything fails.
- The session booking is cleared after success.
- The calendar and resend buttons do real work.

TESTS

- Double booking blocked at the database level, cancelled slot can be rebooked, atomic rollback on failure, another user's appointment returns 404, anonymous access redirects to login.

NOTES FOR THE TEAM

- The notification service is the only place that sends messages. Views call it, they never talk to a provider directly.

Stage 7: Live queue, polling, cancel, staff call-next

NOT STARTED

GOAL

Show the student where they are in line in real time, let them cancel, and let staff call patients from admin.

WHAT TO BUILD

- **Queue state is calculated** from Appointment rows (serving number, students ahead, estimated wait by priority). Nothing is stored twice.
- `/queue/` renders the page. `/queue/status/` returns JSON. A small JS module polls it about every 10 seconds and updates the page. No WebSockets.
- **Cancel**: POST to `/appointments/<ref>/cancel/` from the prototype's confirmation modal. Free up to 15 minutes before the slot. The slot becomes bookable again.
- **Reschedule**: cancel plus a new booking flow with the practitioner preselected.
- **Staff**: Django admin actions to call the next patient, mark checked in, completed or no-show. Calling a patient sends a notification.
- **Remove the prototype's fake 8-second automatic call**. It was demo scaffolding.

DONE WHEN

- A student sees their position change after staff act in admin, without reloading.
- A student can only see and cancel their own appointment.

TESTS

- Queue calculation for each priority, JSON endpoint permissions, cancel rules (allowed, too late, already cancelled), cancelled slot reusable, call-next action updates status and creates a notification.

NOTES FOR THE TEAM

- Polling should pause when the browser tab is hidden. It is a small saving that matters on shared campus machines.

Stage 8: History, dashboard, notifications, help

NOT STARTED

GOAL

Replace every remaining sample value with real data.

WHAT TO BUILD

- **/history/**: real query on Appointment with the prototype's filters. Clinic stats (total visits, average wait, high-priority count) computed from the data. Follow-up card.
- **Dashboard**: next appointment, recent visits and quick tiles from real data. Empty state when there is no upcoming visit.
- **Notifications**: Notification model, real dropdown with unread badge, mark as read.
- **Help**: FAQ model with admin, so staff edit the answers. Get directions, call the desk and contact support become real (map link, tel: link, support request).
- Export (download my data) now includes appointments and notifications.
- Delete `apps/core/mock.py` and the placeholder views.

DONE WHEN

- No page reads sample data any more.
- Nothing imports `mock.py`.

TESTS

- History filters, stats maths, dashboard with and without an upcoming appointment, notification read and unread, FAQ ordering, export contents, every query scoped to the signed-in student.

NOTES FOR THE TEAM

- The prototype's 404 sidebar item was demo scaffolding. It does not come back as navigation.

Stage 9: Error pages, security, tests, production

NOT STARTED

GOAL

Make the app safe and ready to deploy.

WHAT TO BUILD

- Custom 404, 403 and 500 pages in the prototype's style.
- **Production settings**: allowed hosts and secret key from the environment, secure cookies, HTTPS redirect, HSTS, static files served by WhiteNoise, Gunicorn.
- **Security sweep**: every view requires login, every appointment query is filtered by user, CSRF on every POST, no unsafe HTML rendering of user text, admin reachable only by staff.
- **Test sweep**: triage scoring, slot generation, double booking and permissions are required. Add anything the earlier stages missed.
- README with setup steps. Optional Dockerfile and compose file for deployment.

DONE WHEN

- `manage.py check --deploy` is clean with production settings.
- A new teammate can set the project up from the README alone.

TESTS

- Permission matrix test (anonymous, student, other student, staff) across every URL. Error pages render with the right status code.

NOTES FOR THE TEAM

- Do not run `seed_demo` on a public server. It creates an account with a known password.

6. Prototype bugs and where they get fixed

Bug in the prototype	Fix	Stage
Buttons emit btn-bp but the CSS expects .bp	One naming scheme: btn btn--primary and similar modifiers	1 (done)
.cnt is used for two unrelated things	Split into separate classes	1 (done)
cancelprio can cancel the previous confirmed appointment instead of discarding the draft	Discard only clears the session. Cancel only touches an appointment found by reference and owner	4 and 7
calendarPlus icon does not exist and falls back to the wrong icon	Use the existing calplus icon	1 (done)
404 item in the navigation is demo scaffolding	Removed. Real error pages instead	8 and 9
Fake automatic queue call after 8 seconds	Removed. Staff call the next patient from admin	7

7. What each prototype piece becomes

Prototype	Django
Hardcoded student	User and StudentProfile
PRAC (practitioner list)	Practitioner records
REASONS and RW (weights)	VisitReason records with triage weights
FLAGS and RED	Symptom records with a red-flag field
Fake availability	PractitionerAvailability plus generated slots
HIST	Appointment queryset
S.notifs	Notification model
FAQS	FAQ model
S.pro	StudentProfile
S.set	UserSettings
S.appt and S.queue	Appointment plus calculated queue state
Hardcoded TODAY	Django local date in Asia/Manila
Global state object and render()	Django templates, with small JS modules for UI behaviour
Toast system	Django messages

8. Out of scope for now

- Custom staff dashboard (admin is the staff interface).
- Public signup.
- A real slot reservation lock (the hold is display-only).
- Real SMS provider (console output in development).
- WebSockets, DRF, Celery, Channels, Tailwind, HTMX.

9. Running the project locally

Open a terminal in the project folder:

```
python -m venv .venv
.venv\Scripts\activate (Windows) or source .venv/bin/activate
pip install -r requirements/dev.txt
copy example.env .env (then set a new SECRET_KEY)
python manage.py migrate
python manage.py seed_demo
python manage.py createsuperuser
python manage.py runserver
python manage.py test apps
```

The seed command creates the demo student **alex.mendoza**. Use your own superuser for /admin/.